Vel Tech Group of Institutions

Name: Gontu Yaswanth
Email: vtu33434@veltech.edu.in
Roll no: Vtu33434
Phone: null
Branch: VTU
Department: CSE
Batch: 2028
Degree: B.Tech - Computer Science and Engineering

Scan to verify results

2028_NeoColab_Competitive Coding II_L8

VTU_Week 2_CY

Attempt : 1
Total Mark : 40
Marks Obtained : 40

Section 1 : Coding

1. Problem Statement

Karthik is organizing marbles of three different colors: red, white, and blue. Each color is represented by an integer—0 for red, 1 for white, and 2 for blue. Karthik wants to sort the marbles so that all marbles of the same color are adjacent, and they are arranged in the order red, white, and blue. Help Karthik solve this problem efficiently using the two-pointer technique.

Input Format

The first line of input consists of an integer n, representing the number of marbles in the array nums[].

The second line contains n integers, representing the marbles' colors, where each integer is either 0, 1, or 2.

Output Format

The output prints the sorted array, where all marbles are grouped by color in the order red (0), white (1), and blue (2).

Refer to the sample output for formatting specifications.

Sample Test Case

Input: 6
2 0 2 1 1 0

Output: 0 0 1 1 2 2

Answer

```
// You are using GCC
#include <stdio.h>

void swap(int *a, int *b) {
    int temp = *a;
    *a = *b;
    *b = temp;
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int nums[100];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    int low = 0;      // Pointer for 0s (Red)
    int mid = 0;      // Pointer for 1s (White)
    int high = n - 1; // Pointer for 2s (Blue)

    while (mid <= high) {
        if (nums[mid] == 0) {
            // Found a 0, move it to the 'low' section
            swap(&nums[low], &nums[mid]);
            low++;
            mid++;
```

```
        } else if (nums[mid] == 1) {
            // Found a 1, it's already in the right place relative to mid
            mid++;
        } else {
            // Found a 2, move it to the 'high' section
            swap(&nums[mid], &nums[high]);
            high--;
            // Note: Don't increment mid here because the swapped element
            // from 'high' needs to be checked.
        }
    }

    // Print sorted array with specific spacing
    for (int i = 0; i < n; i++) {
        printf("%d%s", nums[i], (i == n - 1 ? "" : " "));
    }
    printf("\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

2. Problem Statement

Kavya is an avid reader who loves finding hidden patterns in texts. One day, she stumbled upon a string of characters and wondered about the longest palindromic substring it contains. Help Kavya identify this substring using an efficient algorithm based on the two-pointer technique.

Input Format

The input consists of a single string s.

Output Format

The output displays "Longest Palindromic Substring: " followed by the longest palindrome found.

Refer the sample output for format specifications.

Sample Test Case

Input: babad

Output: Longest Palindromic Substring: bab

Answer

```
// You are using GCC
#include <stdio.h>
#include <string.h>

int main() {
    char s[21]; // Max length 20 + null terminator
    if (scanf("%s", s) != 1) return 0;

    int n = strlen(s);
    if (n == 0) return 0;

    int start = 0, maxLen = 1;

    for (int i = 0; i < n; i++) {
        // Case 1: Odd length (center is one character)
        int l = i, r = i;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--;
            r++;
        }

        // Case 2: Even length (center is between two characters)
        l = i, r = i + 1;
        while (l >= 0 && r < n && s[l] == s[r]) {
            if (r - l + 1 > maxLen) {
                start = l;
                maxLen = r - l + 1;
            }
            l--;
            r++;
        }
```

```
    }

    // Output strictly matching the format
    printf("Longest Palindromic Substring: ");
    for (int i = start; i < start + maxLen; i++) {
        printf("%c", s[i]);
    }
    printf("\n");

    return 0;
}
```

Status : Correct **Marks : 10/10**

3. Problem Statement

Rohit is analyzing an array of integers to find special combinations of numbers. Using the Two-Pointer technique, he wants to identify all unique triplets in the array whose sum is equal to 0. Given an integer array, Rohit must find and print all such triplets, ensuring that no duplicate triplets are included in the result.

Write a program to ensure that all valid triplets with sum zero are printed correctly.

Example 1

Input:

6

-1 0 1 2 -1 -4

Output:

-1 -1 2

-1 0 1

Explanation:

nums[0] + nums[1] + nums[2] = (-1) + 0 + 1 = 0.

nums[1] + nums[2] + nums[4] = 0 + 1 + (-1) = 0.

nums[0] + nums[3] + nums[4] = (-1) + 2 + (-1) = 0.

The distinct triplets are [-1,0,1] and [-1,-1,2].

Notice that the order of the output and the order of the triplets does not matter.

Example 2

Input:

3

0 1 1

Output:

No triplets found with sum 0.

Explanation:

The only possible triplet does not sum up to 0.

Example 3

Input:

3

0 0 0

Output:

0 0 0

Explanation:

The only possible triplet sums up to 0.

Input Format

The first line contains an integer n, representing the number of elements in the array.

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints each unique triplet whose sum is 0 on a separate line.

Each triplet must be printed in ascending order (as they appear after sorting).

If no such triplets exist, print "No triplets found with sum 0."

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6
-1 0 1 2 -1 -4

Output: -1 -1 2
-1 0 1

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>

// Comparison function for qsort
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 0;

    int nums[n];
    for (int i = 0; i < n; i++) {
        scanf("%d", &nums[i]);
    }

    // Sorting is essential for the two-pointer approach and duplicate handling
    qsort(nums, n, sizeof(int), compare);
```

```
    int found = 0;

    for (int i = 0; i < n - 2; i++) {
        // Skip duplicate values for the first element
        if (i > 0 && nums[i] == nums[i - 1]) continue;

        int left = i + 1;
        int right = n - 1;

        while (left < right) {
            int sum = nums[i] + nums[left] + nums[right];

            if (sum == 0) {
                printf("%d %d %d \n", nums[i], nums[left], nums[right]);
                found = 1;

                // Move pointers and skip duplicates for left and right elements
                while (left < right && nums[left] == nums[left + 1]) left++;
                while (left < right && nums[right] == nums[right - 1]) right--;

                left++;
                right--;
            } else if (sum < 0) {
                left++; // Sum is too small, move left pointer to increase sum
            } else {
                right--; // Sum is too large, move right pointer to decrease sum
            }
        }
    }

    if (!found) {
        printf("No triplets found with sum 0.\n");
    }

    return 0;
}
```

Status : Correct **Marks : 10/10**

4. Problem Statement

Rahul is given an array of integers and a target value T. Using the Two-Pointer technique, he wants to find two distinct elements in the array such that the sum of the pair is closest to the target value. To apply this technique efficiently, the array must first be sorted.

Write a program to ensure that the pair of numbers whose sum is closest to T is identified and printed correctly.

Input Format

The first line contains two space-separated integers n and T, where:

- n is the number of elements in the array
- T is the target sum

The second line contains n space-separated integers, representing the elements of the array.

Output Format

The output prints two space separated integers representing the pair whose sum is closest to the target.

The output pair must be printed in ascending order.

If multiple pairs have the same closest sum, print the first pair found by the two-pointer traversal.

Refer to the sample output for the formatting specifications.

Sample Test Case

Input: 6 50
10 22 28 29 30 40

Output: 10 40

Answer

```
// You are using GCC
#include <stdio.h>
#include <stdlib.h>
```

```c
#include <limits.h>

// Comparison function for qsort to sort the array in ascending order
int compare(const void *a, const void *b) {
    return (*(int*)a - *(int*)b);
}

int main() {
    int n, T;

    // Read n and target sum T
    if (scanf("%d %d", &n, &T) != 2) return 0;

    int arr[10]; // Constraint: n <= 10
    for (int i = 0; i < n; i++) {
        scanf("%d", &arr[i]);
    }

    // Step 1: Sort the array as required by the Two-Pointer approach
    qsort(arr, n, sizeof(int), compare);

    int left = 0;
    int right = n - 1;
    int res_l, res_r;
    long min_diff = LONG_MAX;

    // Step 2: Apply Two-Pointer technique
    while (left < right) {
        int current_sum = arr[left] + arr[right];
        long current_diff = labs((long)current_sum - T);

        // Update the closest pair if a smaller difference is found
        if (current_diff < min_diff) {
            min_diff = current_diff;
            res_l = arr[left];
            res_r = arr[right];
        }

        // Move pointers based on current sum vs Target
        if (current_sum < T) {
            left++;
        } else if (current_sum > T) {
```

```
            right--;
        } else {
            // If sum == T, it's the closest possible match
            break;
        }
    }

    // Step 3: Print the pair in ascending order
    printf("%d %d", res_l, res_r);

    return 0;
}
```

Status : Correct **Marks : 10/10**